Boot: a Small Self-Hosting Language for VM16

Specification, version 0.1 (draft)

November 4, 2026

Abstract

Boot is a small imperative language with dynamic types, C-style braces, and one data structure: the dictionary. It is designed for one purpose: to compile itself on VM16, running on a real Z80 or 6502 with 64 KB of memory. This document defines its syntax (in BNF), its values and their meaning, the representation of values and programs on VM16, and the plan for bootstrapping the compiler.

1 Goals

1. **Self-hosting on 8-bit hardware.** The compiler is written in Boot, and it compiles itself on VM16 within 64 KB. Every decision below is measured against this.
2. **Simple to compile.** A one-pass recursive-descent compiler, with no syntax tree, emits VM16 code directly, in the tradition of PL/0, Small-C and BCPL.
3. **Pleasant enough to write a compiler in.** Strings, dictionaries, recursion and character I/O; no type declarations.

Non-goals: speed, floating point, closures, a large library, and integers wider than 15 bits.

2 A first example

```
// Count the words on standard input, and print each with its count.
var counts = {};

fun word() {                      // the next word, or nil at end of input
    var c = getc();
    while (c == ' ' || c == '\n') { c = getc(); }
    if (c < 0) { return nil; }
    var buf = [];                 // a dictionary with keys 0, 1, 2, ...
    while (c >= 0 && c != ' ' && c != '\n') {
        buf[len(buf)] = c;
        c = getc();
    }
    return str(buf);
}

fun main() {
    var w = word();
    while (w != nil) {
        if (counts[w] == nil) { counts[w] = 0; }
        counts[w] = counts[w] + 1;
        w = word();
```

```
    }
    for (w in counts) { print(w); putc('␣'); print(counts[w]); putc('\n'); }
}
```

A program is a series of global variable and function definitions. Running it first defines every function, then initialises the global variables in order, and then calls `main()`. So an initialiser may call any function, but it sees a global variable declared after it as `nil`.

3 Lexical structure

token	form
name	a letter or `_`, then letters, digits and `_`
integer	decimal digits, or `0x` and hexadecimal digits; at most `0x7FFF`, taken modulo 2^{15}
character	`'a'`: one character or escape between single quotes; its value is an integer
string	`"text"`: characters and escapes between double quotes
escape	`\n \t \r \0 \\ \' \"` `\x`*hh*
keyword	`fun var if else while for in return break continue nil true false`
operator	`( ) { } [ ] , ; : . = == != < <= > >= + - * / % ! && ||`
comment	`//` to the end of the line

Spaces, tabs and newlines separate tokens and have no other meaning. Names are case-sensitive.

4 Grammar

In the grammar, $\{\, x \,\}$ means zero or more x, $[\, x \,]$ means an optional x, and quoted text is a token. Precedence is encoded in the grammar, loosest first; all binary operators are left-associative except the comparisons, which do not associate (`a < b < c` is an error). The whole grammar is in fig. 1.

Notes.

- Braces are required around every body, so there is no dangling `else`. A *block* as a statement and a *dict* as an expression never meet: a statement cannot start with `"{"` as an expression.
- In *expr* `"="` *expr*, the left side must be a NAME, or a *postfix* ending in `"["..."]"` or `"."`NAME. Assignment is a statement, not an expression.
- The grammar is LL(1): one token of lookahead decides every choice. Whether an expression statement is an assignment is only known at the `"="`; the compiler keeps the code for the last indexing step pending until then (section 8).

5 Values and meaning

Every value has one of five types:

type	values
nil	the single value `nil`: the value of a missing key and of an uninitialised variable
integer	-16384 to 16383. Arithmetic wraps modulo 2^{15}. `true` and `false` are the integers 1 and 0
string	an immutable sequence of bytes. Strings are *interned*: two strings with the same bytes are the same object
dictionary	a mutable map from keys (integers and strings) to values other than `nil`
function	a function defined with `fun`

```
program  ::= { global }
global   ::= vardecl | fndef
fndef    ::= "fun" NAME "(" [ params ] ")" block
params   ::= NAME { "," NAME }
vardecl  ::= "var" NAME [ "=" expr ] ";"

block    ::= "{" { stmt } "}"
stmt     ::= vardecl | block | if
           | "while" "(" expr ")" block
           | "for" "(" NAME "in" expr ")" block
           | "return" [ expr ] ";" | "break" ";" | "continue" ";"
           | expr [ "=" expr ] ";"
if       ::= "if" "(" expr ")" block [ "else" ( block | if ) ]

expr     ::= and { "||" and }
and      ::= compare { "&&" compare }
compare  ::= sum [ ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) sum ]
sum      ::= product { ( "+" | "-" ) product }
product  ::= unary { ( "*" | "/" | "%" ) unary }
unary    ::= ( "-" | "!" ) unary | postfix
postfix  ::= primary { "(" [ args ] ")" | "[" expr "]" | "." NAME }
args     ::= expr { "," expr }
primary  ::= NAME | INTEGER | CHARACTER | STRING | "nil" | "true" | "false"
           | "(" expr ")" | dict | list
dict     ::= "{" [ entry { "," entry } ] "}"
entry    ::= ( NAME | STRING | INTEGER | CHARACTER ) ":" expr
list     ::= "[" [ args ] "]"
```

Figure 1: The grammar of Boot.

Truth. `nil` and 0 are false; every other value is true. `!`, `&&`, `||` and the comparisons return 1 or 0, and `&&` and `||` evaluate their right operand only when needed.

Operators.

- `+ - * / %` take integers; `/` and `%` truncate towards zero, as in C, and division by zero is an error. `+` also concatenates two strings.
- `< <= > >=` compare two integers, or two strings in byte order.
- `==` and `!=` take any two values. Integers are equal when their values are; strings are equal when their bytes are, which by interning is the same as being the same object; dictionaries and functions are equal only to themselves.

Any other combination of operand types is a run-time error.

Dictionaries. `d[k]` is the value stored under key `k`, or `nil`. `d[k] = v` stores it, and storing `nil` removes the key. `d.name` is short for `d["name"]`. A key must be an integer or a string. `{a: 1, "b c": 2, 7: 3}` creates a dictionary; a NAME before `":"` is a string key. `[x, y, z]` is short for `{0: x, 1: y, 2: z}`.

Iteration. `for (k in d) {...}` binds the local variable `k` to each key of `d` in turn: first the integer keys $0, 1, 2, \ldots$ up to the first missing one, in increasing order, then the remaining keys in the order they were first inserted. The order is therefore the same on every implementation and every run, which the bootstrap test (section 9) depends on. Adding or removing keys of `d` inside the loop is an error, or undefined if the implementation does not check it.

Strings. `s[i]` is the byte at position `i` (from 0) as an integer, or `nil` outside the string. Strings cannot be changed.

Variables and scope. A `var` at the top level declares a global; one inside a function declares a local, visible from its declaration to the end of the enclosing block. Parameters are locals. A name that is not a local refers to a global, which may be declared later in the program, so functions can call each other in any order. Every global that is used must be declared somewhere, and no name may be declared twice in the same scope.

Functions. Functions are values: they can be stored in variables and dictionaries and called through them, `ops.add(x, y)`. They are defined only at the top level, so there are no closures. Calling a value that is not a function, or with the wrong number of arguments, is an error. A function that ends without `return` returns `nil`. Arguments are evaluated left to right.

Errors. A run-time error prints a message and stops the program with exit status 1. There is no way to catch an error.

6 Built-in functions

The built-ins are global functions with reserved names.

function	result
`getc()`	the next byte of standard input, or -1 at end of input
`putc(c)`	writes the byte c mod 256
`putw(n)`	writes n mod 2^{15} as a little-endian word, so that a compiler can write addresses up to `0x7FFF`
`print(v)`	writes an integer in decimal, or a string's bytes
`len(v)`	the number of bytes of a string, or of keys of a dictionary
`str(d)`	the string whose bytes are `d[0]`, `d[1]`, ... up to the first missing key
`chr(c)`	the one-byte string with byte c
`type(v)`	`"nil"`, `"int"`, `"str"`, `"dict"` or `"fun"`
`error(s)`	prints `s` and stops with an error
`exit(n)`	stops with exit status `n`

7 Representation on VM16

This section is not part of the language; it is the plan for the first implementation.

Values are single 16-bit words, distinguished by their low bit and their address:

word	value
$2n + 1$ (odd)	the integer n
0	`nil`
even, nonzero	the address of an object, whose first word gives its type

Integer arithmetic on tagged words is cheap: $a + b$ is $a + b - 1$, and a comparison of two integers is a comparison of the words. This is the scheme of the CAM and of the graph reducer in this repository.

Objects. Every object starts with a header word holding its type in the low 3 bits and a size above them. A *string* holds its length, a hash and its bytes. A *dictionary* holds a count, an *array part* for the keys $0, 1, \ldots, n-1$ (one word per value), and a *hash part*: an array of entries (key, value) in insertion order, with an index of small integers into it, as in CPython's compact dictionaries. The array part is what makes a dictionary used as an array cost a word per element, and the two parts together give the iteration order of section 5. A *function* holds its number of parameters, followed by its code.

Static objects. String literals and functions are placed in the code area by the compiler and are never freed. Objects outside the heap are ignored by the collector, which therefore only needs to test whether a word is an even address inside the heap.

Interning. The runtime keeps a hash set of all strings. A string is only created through it, so two equal strings are one object. The set is weak: the collector removes strings that are not otherwise reachable.

Garbage collection. Mark and sweep, not copying: a copying collector would halve the usable heap. The roots are the data stack and the global table. Free blocks are kept on a free list and joined during the sweep.

Frames. Arguments are pushed on the data stack left to right, and `CALL` pushes the return address on the separate return stack, so the data stack holds only values and saved frame pointers, which are never heap addresses. The callee saves the frame pointer (a fixed register), pushes `nil` for each of its locals, and addresses parameters and locals as `[fp±k]`. The result is returned in r_0.

Code. The compiler generates code for a stack machine on VM16's data stack and calls runtime routines for everything that checks types: arithmetic, comparison, indexing, calls through values. Compiled code is larger and slower than it could be, but the compiler stays simple and the runtime is shared.

8 Programs and the compiler

The object file. A compiled program is not a VM16 image. The runtime `rt.v16` is the image: it reads the compiled program from the start of standard input, places it in memory, and runs it; the rest of standard input is the program's own input. This is how the graph reducer reads its `.graph` files, and it means one fixed image on the target machine, written in VM16 assembly:

```
cat prog.bto input.txt | vm16 rt.v16
```

An object file (`.bto`) is a sequence of chunks, each a kind byte, a length word and that many bytes:

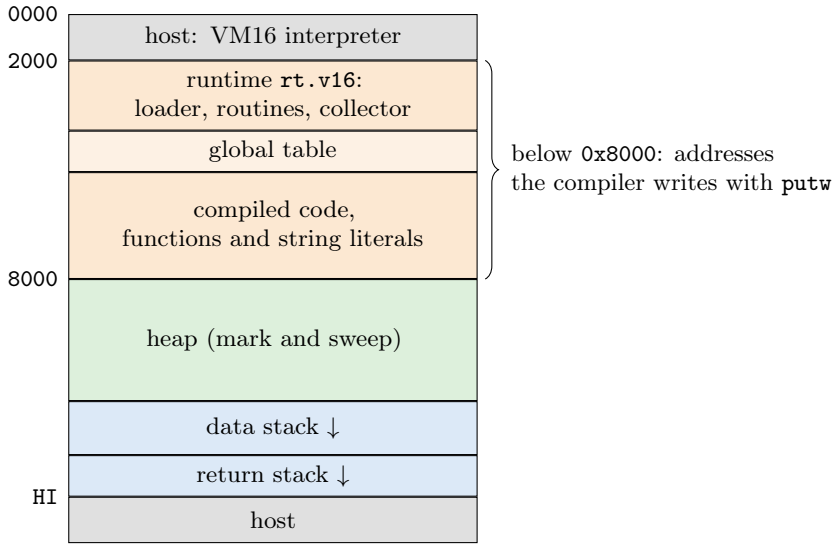


Figure 2: The planned memory map. `0x2000` is where the Z80 build of VM16 loads images; HI is the top of the usable region from `SYS 2`.

kind	meaning
`C`	code: copied to the next free address of the code area
`S`	a string literal: copied, and entered in the intern set
`E`	the end; the body is the entry address, the initialisation code

The compiler keeps the same running address as the loader, so it knows where each chunk will land.

The compiler reads the source once, with one token of lookahead, and has no syntax tree: each parsing function emits code as it recognises its construct.

- The code for one function is collected in a dictionary of bytes (its array part), so that jumps within the function can be patched once their targets are known. At the end of the function it is written out as one `C` chunk and the dictionary is dropped. No more than one function's code is ever in memory.
- An expression statement is compiled as an expression until the compiler sees whether `"="` follows. The last step of a *postfix*, `[k]` or `.name`, is therefore not emitted at once: the compiler emits the code for the dictionary and the key, and remembers that a *get* is pending. If `"="` follows, it emits the value and a *set* instead; otherwise it emits the *get*. A plain NAME is handled the same way.
- Globals are numbered in order of first use, and each has a slot in the global table. A call to a function, even one not yet defined, loads its global slot and calls through it, so no call ever needs patching.
- Each string literal is written once, as an `S` chunk, the first time it is used. Because the loader interns these before the program runs, the literal is the string every later equal string will be.
- The stores of each function into its slot and the initialisers of the top-level `var`s are collected as they are read, and written at the end: first all the function stores, then the initialisers in program order, then a call to `main` and a halt. That is the `E` chunk.

At the end, the compiler reports any global that was used and never declared.

9 Bootstrapping

1. Write `rt.asm`, the runtime, in VM16 assembly, and `btc.py`, a compiler from Boot to object files, in Python. `btc.py` is the reference, and is written in the style of Boot: one pass, no syntax tree, dictionaries only, so that it can be translated by hand almost line by line.
2. Write `btc.bt`, the same compiler in Boot, and test both on the same programs. Their output must be identical, byte for byte.
3. Compile `btc.bt` with `btc.py`, giving `btc1.bto`. Use it to compile `btc.bt` again:

   ```
   python3 btc.py btc.bt -o btc1.bto
   cat btc1.bto btc.bt | vm16 rt.v16 > btc2.bto
   cmp btc1.bto btc2.bto
   ```

   The two files must be identical: the compiler has reached a fixpoint.
4. Repeat the last step on the Z80 build of VM16. From then on `rt.v16` and `btc1.bto` are all the target needs.

10 Open questions

- **Will it fit?** The code area below `0x8000` must hold the runtime, the global table and the compiled compiler; the heap above it the compiler's symbol table and one function's code. The first measurement should be the size of the compiled compiler and its peak heap use, under `btc.py` and `vm16 -c`, long before the Z80.
- **15-bit integers.** Addresses from `0x4000` to `0x7FFF` appear as negative integers. Adding offsets to them works, because arithmetic wraps, but comparing them does not; the compiler should compare offsets, not addresses.
- **Bit operations.** There are none: `/` and `%` are enough to split values into bytes. A compiler for an instruction set with bit fields would want `&`, `|` and shifts.
- **Strings.** Is `str`, `chr` and `+` enough string building? Each concatenation creates and interns a new string.
- **Closures and nested functions** are left out on purpose; they would need environments on the heap.